⌂ Volver al inicio

Índice

1. Uso de estructuras de control
2. Selección y estructuras de selección
    - Actividades 2
3. Repetición y estructuras de repetición
    - Actividades 3
    - Actividades extra
4. Estructuras de salto
5. Documentación de programas
    - Actividades 5

---

1. Uso de estructuras de control

⬆ Volver al índice

Las estructuras de control determinan el orden en que se ejecutan las instrucciones. Sin ellas, un programa solo avanzaría de arriba abajo una vez.

Las tres familias principales son:

- **Secuencia:** instrucciones en orden.
- **Selección:** elegir entre caminos.
- **Repetición:** ejecutar un bloque varias veces.

Estas familias no son alternativas aisladas. Un programa real suele combinar las tres: primero recibe y prepara unos datos, después toma decisiones sobre ellos y, por último, repite una operación para cada elemento o hasta que se cumpla una condición. La estructura elegida debe corresponder al problema que se quiere resolver, no a una preferencia de escritura.

Un esquema general de un programa puede representarse así:

```
ENTRADA DE DATOS
       ↓
SECUENCIA DE PASOS
       ↓
DECISIONES
       ↓
ACCIONES REPETIDAS, SI SON NECESARIAS
       ↓
RESULTADO
```

En Python, cada bloque debe quedar claramente separado del siguiente. La indentación será la forma concreta de marcar esos bloques cuando se estudie la sintaxis, pero en esta introducción basta con entender que las instrucciones que pertenecen a una misma decisión o repetición forman un grupo.

1.1 Secuencia

La secuencia es el flujo más sencillo: cada paso se realiza una vez y en el orden en que aparece. El resultado de un paso puede convertirse en la entrada del siguiente. Por eso el orden sí importa; no es equivalente calcular un total antes o después de aplicar un descuento.

Un esquema de secuencia sería:

```
RECIBIR EL NOMBRE
       ↓
PREPARAR EL SALUDO
       ↓
MOSTRAR EL SALUDO
```

La secuencia es suficiente cuando no hay alternativas ni acciones que repetir. Por ejemplo, un programa que pide un nombre, prepara un mensaje y lo muestra solo necesita seguir esos pasos consecutivamente.

1.2 Condición como expresión booleana

Una condición es una pregunta cuya respuesta solo puede ser afirmativa o negativa. El programa utiliza esa respuesta para decidir si continúa por un camino o por otro. Las preguntas pueden comparar cantidades, comprobar si un dato está disponible o verificar si una situación es válida.

Esquema de una decisión sencilla:

```
COMPROBAR UNA CONDICIÓN
          ↓
   ¿SE CUMPLE?
     ↙     ↘
   SÍ       NO
   ↓        ↓
REALIZAR  CONTINUAR
ACCIÓN    SIN ESA ACCIÓN
```

Por ejemplo, antes de retirar dinero se puede comprobar si el saldo disponible es suficiente. Si lo es, se realiza la retirada; si no, se informa de que la operación no puede hacerse. La comprobación debe realizarse antes de la acción que depende de ella.

1.3 Repetición controlada por colección

La repetición controlada por una colección se utiliza cuando una misma acción debe realizarse para cada elemento de un conjunto de datos. El programa toma un elemento, realiza la acción, pasa al siguiente y termina cuando ya no quedan elementos.

```
OBTENER LA COLECCIÓN
          ↓
TOMAR EL SIGUIENTE ELEMENTO
```

```
          ↓
REALIZAR LA ACCIÓN
          ↓
¿QUEDAN MÁS ELEMENTOS?
      ↙           ↘
    SÍ             NO
    ↑              ↓
VOLVER AL PASO   TERMINAR
ANTERIOR
```

Este modelo evita repetir manualmente las mismas instrucciones y sigue funcionando aunque la colección tenga más o menos elementos.

1.4 Repetición controlada por condición

La repetición controlada por una condición se utiliza cuando no se conoce de antemano cuántas veces debe repetirse una acción. Antes de cada vuelta se comprueba si se mantiene la situación que permite continuar.

```
COMPROBAR LA CONDICIÓN
          ↓
   ¿SE PUEDE CONTINUAR?
      ↙           ↘
    SÍ             NO
    ↓              ↓
REALIZAR        TERMINAR
LA ACCIÓN
    ↑
    └── ACTUALIZAR LOS DATOS
```

Es necesario que cada vuelta pueda cambiar el resultado de la comprobación. Si la condición nunca cambia, se produce un bucle infinito. Este modelo es apropiado para menús, reintentos y procesos que terminan cuando el usuario introduce una respuesta concreta o cuando se alcanza un estado determinado.

1.5 Diseñar antes de codificar

Antes de escribir estructuras de control conviene responder:

1. ¿Qué datos entran?
2. ¿Qué resultado debe producirse?
3. ¿Qué decisiones existen?
4. ¿Qué acciones se repiten?
5. ¿Cuándo termina el programa?
6. ¿Qué pasa si la entrada no es válida?

Un esquema sencillo para clasificar una nota sería:

```
LEER LA NOTA
      ↓
```

```
¿ESTÁ ENTRE 0 Y 10?
   ↙           ↘
 NO             SÍ
 ↓              ↓
AVISAR        ¿ALCANZA EL APROBADO?
DEL ERROR       ↙          ↘
              SÍ            NO
              ↓             ↓
          APROBADO       SUSPENSO
```

Este diseño separa los casos y hace visibles las reglas del problema. Después se puede traducir a Python, comprobar cada camino y añadir validaciones sin tener que improvisar la lógica mientras se escribe. También conviene preparar casos de prueba para una nota inválida, una nota exactamente igual al límite del aprobado, una nota intermedia y la nota máxima.

---

2. Selección y estructuras de selección

⬆ Volver al índice

Las estructuras de selección ejecutan uno u otro bloque según una condición. Permiten representar decisiones del tipo «si ocurre esto, haz una cosa; en caso contrario, haz otra». La condición debe expresar una pregunta clara sobre los datos del programa y no mezclar cambios de estado innecesarios.

Python determina el bloque que se ejecuta siguiendo estas reglas:

1. Evalúa la condición del `if`.
2. Si es verdadera, ejecuta ese bloque y omite los bloques alternativos.
3. Si es falsa, prueba el siguiente `elif`, si existe.
4. Si ninguna condición es verdadera, ejecuta `else`, si se ha escrito.

En todos los casos se ejecuta como máximo una rama de una misma cadena `if`/`elif`/`else`.

2.1 `if`

`if` sirve para ejecutar instrucciones solo cuando se cumple una condición. No es necesario escribir un `else` si no hay ninguna acción que realizar cuando la condición sea falsa.

```python
temperatura = 28

if temperatura > 25:
    print("Hace calor")
```

Si la temperatura fuese `25` o menor, no se mostraría ningún mensaje y el programa continuaría después del bloque. Los dos puntos indican que comienza un bloque y la indentación indica qué instrucciones forman parte de él.

Un `if` es adecuado cuando solo interesa actuar en el caso afirmativo. Por ejemplo, se puede mostrar una advertencia si queda poco espacio en disco, pero no hacer nada especial cuando el espacio disponible es

suficiente. No conviene añadir un `else` vacío: si no hay una respuesta para el caso contrario, es más claro dejarlo sin escribir.

La condición se evalúa con el valor que tienen las variables en ese momento. Si una variable cambia después, esa modificación no cambia una decisión que ya se ha tomado. Por eso las comprobaciones deben colocarse justo antes de la acción que protegen y deben utilizar datos que estén actualizados.

2.2 `if ... else`

`else` define el camino alternativo. Se utiliza cuando deben cubrirse exactamente dos posibilidades, como par/impar, válido/no válido o disponible/no disponible:

```
numero = 7

if numero % 2 == 0:
    print("Es par")
else:
    print("Es impar")
```

El operador `%` calcula el resto de la división. Si el resto de dividir entre dos es cero, el número es par; si no, es impar. La condición no compara directamente con las palabras «par» o «impar», sino que deduce la respuesta a partir del dato.

`else` no lleva una condición propia: representa todos los casos en los que el `if` no se ha cumplido. Por ese motivo es útil para garantizar una respuesta completa. En este ejemplo, cualquier entero queda clasificado, porque todo número es par o impar. Antes de utilizar `else`, conviene comprobar que el caso alternativo realmente incluye todos los valores restantes y que no se están ocultando entradas inválidas.

Si el programa acepta datos introducidos por una persona, la comprobación de paridad debería hacerse después de validar que el dato es numérico. La selección decide qué hacer con un dato válido; no sustituye por sí sola la validación del formato de entrada.

2.3 `if ... elif ... else`

Cuando hay más de dos resultados posibles se encadenan varias condiciones con `elif`. El orden es fundamental: las condiciones más específicas o exigentes deben aparecer antes que las más generales.

```
nota = 8.4

if nota >= 9:
    calificacion = "sobresaliente"
elif nota >= 7:
    calificacion = "notable"
elif nota >= 5:
    calificacion = "aprobado"
else:
    calificacion = "suspenso"
```

```
print(calificacion)
```

Las condiciones se evalúan de arriba abajo y solo se ejecuta el primer bloque verdadero. Con `nota = 8.4`, la primera comparación es falsa y la segunda es verdadera, por lo que se asigna `notable` y no se revisan las condiciones restantes. Si se colocara primero `nota >= 5`, todas las notas aprobadas entrarían en esa rama y nunca se alcanzarían las categorías superiores.

Una cadena de `elif` funciona bien cuando las categorías son excluyentes y están ordenadas. Los límites deben definirse con cuidado: en el ejemplo, una nota exactamente igual a `7` obtiene `notable` y una exactamente igual a `5` obtiene `aprobado`. Probar los valores situados justo en cada límite ayuda a detectar errores como utilizar `>` cuando se necesitaba `>=`.

`else` puede omitirse si existe un caso que no se desea tratar o si la validación se realiza en otra parte. Sin embargo, incluirlo suele ser conveniente cuando el programa debe ofrecer una respuesta para cualquier dato que llegue hasta esa decisión.

2.4 Condiciones compuestas

Los operadores lógicos permiten combinar comprobaciones:

- `and` exige que todas las condiciones sean verdaderas.
- `or` exige que al menos una sea verdadera.
- `not` invierte el resultado lógico.

```
edad = 25
documento_valido = True

if edad >= 18 and documento_valido:
    print("Acceso permitido")
```

Aquí deben cumplirse simultáneamente la mayoría de edad y la validez del documento. Los paréntesis ayudan a dejar clara la prioridad cuando se mezclan `and` y `or`, por ejemplo:

```
if (es_personal or es_administrador) and cuenta_activa:
    print("Acceso permitido")
```

En la segunda condición, basta con que la persona sea personal o administradora, pero en ambos casos la cuenta debe estar activa. Sin los paréntesis, una expresión con varios operadores puede resultar difícil de leer e interpretar de forma incorrecta al modificarla más adelante.

Python además aplica evaluación en cortocircuito: con `and`, si la primera parte ya es falsa no necesita evaluar la segunda; con `or`, si la primera ya es verdadera tampoco necesita evaluar la segunda. Esta característica puede evitar operaciones innecesarias, pero las condiciones no deberían depender de efectos secundarios ocultos. Es preferible que cada comprobación sea clara y predecible.

2.5 Guard clauses

Una *guard clause* comprueba al principio un caso inválido y termina pronto la función. Así se evita envolver toda la lógica válida dentro de varios niveles de `if` anidados:

```
def procesar_edad(edad):
    if edad < 0:
        return "Edad no válida"
    if edad < 18:
        return "Menor"
    return "Adulto"
```

Cada `return` deja claro qué resultado corresponde a cada caso. Si la función recibe `-1`, no continúa con las comprobaciones posteriores. Esta técnica es especialmente útil para validar parámetros al comienzo de una función.

Las guard clauses no significan que toda función deba tener muchos `return`. Su objetivo es separar pronto los casos excepcionales y dejar la ruta normal sin anidamientos. Una función bien diseñada debe mantener un orden fácil de seguir: primero rechaza los datos imposibles, después comprueba los casos especiales y finalmente ejecuta el caso válido.

También es importante decidir qué ocurre ante un dato incorrecto. La función puede devolver un mensaje, devolver un valor especial o lanzar una excepción, según el contrato del programa. La guard clause organiza la decisión, pero no elige por sí sola la estrategia de comunicación del error.

2.6 `match` y `case`

Desde Python 3.10 se puede hacer coincidencia estructural. `match` compara un valor con varios patrones y ejecuta el primer `case` que coincide:

```
comando = "listar"

match comando:
    case "crear":
        print("Creando")
    case "listar":
        print("Listando")
    case "salir":
        print("Saliendo")
    case _:
        print("Comando desconocido")
```

El patrón `_` actúa como caso por defecto y coincide con cualquier valor que no haya coincidido antes. `match` es útil cuando hay muchos patrones posibles o cuando se comparan estructuras de datos. Para una sola condición booleana, `if` suele ser más claro.

Cada `case` representa una posibilidad concreta del valor que se está analizando. En el ejemplo, el programa no pregunta si el comando es mayor, menor o igual que otro valor, sino que compara su forma con varios

patrones conocidos. El caso `_` evita que una entrada desconocida quede sin respuesta.

`match` no debe utilizarse solo porque el código parezca más moderno. Si las decisiones dependen de intervalos, de varias comprobaciones lógicas o de condiciones que no son patrones concretos, una cadena de `if`/`elif` suele expresar mejor la regla. La elección debe favorecer la lectura del criterio que se está aplicando.

También es posible añadir una **guarda** a un `case` mediante `if`. La guarda se evalúa después de que el patrón haya coincidido, por lo que permite expresar comparaciones como intervalos o combinar varias condiciones:

En cada `case`, `valor` es una variable que captura el valor que se está analizando: aquí, el valor de `numero`. Por tanto, como `numero` vale `12`, la primera guarda comprueba si `12 > 10` y se ejecuta ese bloque. El nombre `valor` podría sustituirse por otro nombre; no es una palabra especial de `match`.

```
numero = 12

match numero:
    case valor if valor > 10:
        print("Mayor que 10")
    case valor if 0 <= valor <= 10:
        print("Entre 0 y 10")
    case _:
        print("Negativo")
```

También se pueden combinar alternativas dentro de un patrón y utilizar una guarda para una comprobación adicional:

```
rol = "editor"
cuenta_activa = True

match rol:
    case "admin" | "editor" if cuenta_activa:
        print("Puede modificar contenido")
    case _:
        print("Acceso limitado")
```

Por tanto, `match` sí puede participar en comprobaciones complejas, pero no conviene forzar toda la lógica dentro de un `case`. Si la decisión consiste principalmente en varias expresiones booleanas independientes, `if`/`elif` normalmente resulta más directo. `match` aporta más claridad cuando primero se quiere identificar el tipo, la forma o el valor de los datos y después aplicar una condición adicional.

2.7 Evitar comparaciones innecesarias

Los valores booleanos ya representan una condición, por lo que no hace falta compararlos explícitamente con `True` o `False`:

```
activo = True
```

```
if activo:
    print("Activo")

# Menos idiomático:
# if activo == True:
#     print("Activo")
```

También se puede comprobar directamente si una colección contiene elementos:

```
usuarios = ["Ana"]
if usuarios:
    print("Hay usuarios para procesar")
```

Una lista vacía, una cadena vacía, `0` y `None` se consideran falsos en una condición. Conviene usar esta característica cuando mejora la legibilidad, pero comparar explícitamente con `None` sigue siendo recomendable:

```
resultado = buscar_usuario()
if resultado is None:
    print("Usuario no encontrado")
```

Estos valores se conocen como valores *falsy*. Muchos valores distintos de ellos se consideran *truthy*, como una lista con elementos, una cadena no vacía o un número distinto de cero. No significa que sean literalmente `True` o `False`, sino que Python puede utilizarlos como resultado práctico de una condición.

Hay que diferenciar entre «no hay valor» y «el valor es falso». Por ejemplo, `0` puede ser una cantidad válida y no debería confundirse automáticamente con un resultado ausente. Cuando esa diferencia importa, conviene comparar de forma explícita y documentar qué representa cada valor.

Actividades 2

Actividad 1: simulación de operaciones bancarias

Simula una única operación de un banco. Parte de una cuenta existente con un titular, un saldo inicial aleatorio entre `0 €` y `1.000 €` y el estado de la cuenta. Solicita la operación que se quiere realizar: `ingresar`, `retirar`, `abrir cuenta`, `cerrar cuenta` o `consultar`.

Cada operación debe producir el resultado adecuado:

- al ingresar, solicita una cantidad positiva y actualiza el saldo;
- al retirar, solicita una cantidad positiva e impide que el saldo quede negativo;
- al abrir cuenta, informa si ya existe una cuenta activa;
- al cerrar cuenta, solo permite el cierre si el saldo es exactamente `0`;
- al consultar, muestra el titular, el saldo y el estado de la cuenta;
- una operación desconocida debe mostrar un aviso claro.

Normaliza el texto de la operación para aceptar mayúsculas y espacios sobrantes. Muestra el estado final de la cuenta tras la operación. Prueba, como mínimo, un ingreso válido e inválido, una retirada con y sin saldo suficiente, un intento de cierre con saldo y otro con saldo `0`. El programa debe resolver una sola operación; no es necesario repetir el menú.

Actividad 2: calcular el precio de una entrada

Una sala de cine vende entradas con estas reglas:

- la entrada general cuesta `8 €`;
- las personas menores de `12` años pagan `5 €`;
- las personas de `65` años o más pagan `6 €`;
- los miércoles se aplica un descuento adicional de `1 €`;
- una sesión en formato 3D añade `2 €`.

Pide la edad, el día de la semana y si la sesión es 3D. Calcula el precio final y muestra el desglose de los importes aplicados. Rechaza edades negativas y decide qué debe ocurrir con una edad que no pueda convertirse a número. Normaliza el día introducido para aceptar mayúsculas y espacios sobrantes. Prueba edades por debajo, justo en y por encima de `12` y `65`, tanto en una sesión normal como en una sesión 3D.

Actividad 3: calculadora con módulos

Crea una calculadora formada por dos archivos:

- `operaciones.py`, que contiene una función independiente para cada operación: sumar, restar, multiplicar y dividir;
- `calculadora.py`, que solicita dos números y la operación deseada, importa las funciones necesarias desde `operaciones.py` y muestra el resultado.

Cada función debe recibir dos valores y devolver el resultado, sin pedir datos ni mostrar mensajes. La división debe controlar el caso de divisor igual a `0` y comunicarlo de forma clara al programa principal. La calculadora debe aceptar las operaciones `+`, `-`, `*` y `/`, además de informar de una operación no reconocida.

El programa principal de `calculadora.py` solo debe ejecutarse cuando ese archivo sea el módulo principal. Prueba cada operación con números enteros y decimales, una división entre cero y un símbolo de operación inválido.

Actividad 4: calcular un envío

Una tienda calcula el coste de envío según el peso del paquete y el tipo de entrega:

- hasta `1 kg`: `3 €`;
- más de `1 kg` y hasta `5 kg`: `6 €`;
- más de `5 kg`: `10 €`;
- la entrega urgente añade `4 €`;
- los pedidos superiores a `50 €` no pagan el envío estándar.

Pide el importe del pedido, el peso y si se desea entrega urgente. Valida que el importe y el peso sean valores no negativos. Muestra el coste de transporte y el total de la compra. Si el pedido supera `50 €`, explica si la

entrega urgente se mantiene como coste adicional. Prueba valores en los límites del peso y del importe, con entrega normal y urgente.

Actividad 5: recomendar un plan de almacenamiento

Una aplicación ofrece tres planes de almacenamiento: `básico`, `profesional` y `empresa`. Pide el plan elegido, el espacio que necesita la persona en gigabytes y si necesita copias de seguridad.

Los planes disponibles ofrecen, respectivamente, `50`, `200` y `1000 GB`. Determina si el plan elegido cubre el espacio solicitado. Las copias de seguridad solo están incluidas en los planes `profesional` y `empresa`; si se solicitan en el plan básico, informa de que es necesario cambiar de plan.

Acepta el nombre del plan con mayúsculas y espacios sobrantes, rechaza planes desconocidos y valida que el espacio solicitado sea un número positivo. Presenta una recomendación final explicando si el plan sirve, si falta espacio o si hay que contratar un plan superior. El programa debe resolver una sola consulta y no necesita repetir el menú ni procesar una colección de consultas.

---

3. Repetición y estructuras de repetición

⬆ Volver al índice

Un bucle ejecuta varias veces un bloque de instrucciones. Cada ejecución se llama **iteración**. En Python hay dos estructuras principales:

- `for`, para recorrer los elementos de una secuencia o repetir una acción un número determinado de veces;
- `while`, para repetir una acción mientras se cumpla una condición.

El bloque que se repite se escribe indentado debajo de la línea del bucle.

3.1 El bucle `for`

El bucle `for` repite un bloque de instrucciones **una vez por cada elemento** de una colección o secuencia. En Python, esta es la forma habitual de recorrer una cadena o una lista. No se indica un contador ni se avanza manualmente: el propio `for` toma el siguiente elemento en cada vuelta.

La forma general es:

```
for variable in secuencia:
    # instrucciones que se repiten
    pass
```

`variable` es un nombre que se elige para representar el elemento actual. `secuencia` es aquello que se quiere recorrer. Las instrucciones indentadas se ejecutan en cada vuelta. Cuando ya no quedan elementos, el bucle termina.

Por ejemplo:

```
for letra in "Sol":
    print(letra)
```

Primero `letra` vale `"S"` y se ejecuta `print(letra)`. Después vale `"o"` y se vuelve a ejecutar la instrucción; por último vale `"l"`. El resultado son tres líneas: `S`, `o` y `l`.

```
frutas = ["pera", "uva", "manzana"]

for fruta in frutas:
    print(fruta)
```

En este caso, `fruta` toma primero el valor `"pera"`, luego `"uva"` y después `"manzana"`. Como la lista tiene tres elementos, el bloque se ejecuta tres veces. Este uso de `for` es parecido al `foreach` de otros lenguajes: se recorren directamente los elementos, sin trabajar con sus posiciones.

Repetir con `range()`

Si no se necesita recorrer una lista concreta y solo se quiere repetir una acción cierta cantidad de veces, se usa `range()`:

```
for numero in range(5):
    print(numero)
```

`range(5)` proporciona los números `0`, `1`, `2`, `3` y `4`; por eso el bloque se ejecuta cinco veces. En cada vuelta, `numero` recibe uno de esos valores. El número final (`5`) no se incluye.

También se pueden indicar el inicio y el final:

```
for numero in range(1, 4):
    print(numero)  # 1, 2, 3
```

Y, si hace falta, cuánto avanzar entre un número y el siguiente:

```
for numero in range(2, 10, 2):
    print(numero)  # 2, 4, 6, 8
```

En `range(inicio, fin, paso)`, el recorrido empieza en `inicio`, avanza según `paso` y se detiene antes de `fin`. Si solo se da el final, se empieza en `0` y se avanza de uno en uno. El paso también puede ser negativo para contar hacia atrás:

```
for numero in range(3, 0, -1):
    print(numero)  # 3, 2, 1
```

Por tanto, `for` y `foreach` no son dos tipos de bucle de Python: Python tiene la instrucción `for`, que recorre los valores que se le proporcionan. Para recorrer los elementos se escribe `for elemento in coleccion`; para repetir un número de veces, se combina con `range()`. La forma `for...of` pertenece a JavaScript, no a Python.

Usar `for` para procesar elementos

Un bucle puede ir procesando los valores y guardar un resultado en una variable. Por ejemplo, para sumar los precios de una lista:

```
precios = [10.5, 8.0, 12.25]
total = 0

for precio in precios:
    total = total + precio

print(total)  # 30.75
```

`total` empieza en `0`; en cada iteración se le añade el precio actual. También se puede escribir `total += precio`.

3.2 El bucle `while`

`while` repite un bloque **mientras** su condición sea verdadera. La condición se comprueba antes de cada iteración:

```
contador = 3

while contador > 0:
    print(contador)
    contador = contador - 1

print("Fin")
```

Se muestran `3`, `2` y `1`. En cada vuelta se resta uno a `contador`; cuando llega a `0`, la condición `contador > 0` es falsa y el bucle termina. Si la condición es falsa desde el principio, el bloque no se ejecuta.

Usa `while` cuando no sepas de antemano cuántas repeticiones harán falta, por ejemplo, si quieres seguir pidiendo una respuesta hasta que sea válida. Es importante que algo dentro del bucle permita que la condición cambie. Si nunca cambia y no hay otra forma de terminar, el programa se queda en un **bucle infinito**.

3.3 Repetir una pregunta o mostrar un menú

Un valor especial puede servir para indicar que se quiere terminar. Se suele llamar **valor de salida** o **centinela**:

```
respuesta = input("Escribe algo (o 'fin' para terminar): ")

while respuesta != "fin":
    print(f"Has escrito: {respuesta}")
    respuesta = input("Escribe algo (o 'fin' para terminar): ")
```

La pregunta aparece antes del bucle y al final de cada iteración. Así se comprueba la respuesta nueva y se puede salir cuando se escribe `"fin"`.

Un menú también puede repetirse usando una condición en el `while`. En este ejemplo, la variable `opcion` se actualiza después de cada respuesta y el menú termina cuando se escribe `"3"`:

```
opcion = ""

while opcion != "3":
    print("1) Saludar")
    print("2) Mostrar ayuda")
    print("3) Salir")
    opcion = input("Elige una opción: ")

    if opcion == "1":
        print("Hola")
    elif opcion == "2":
        print("Selecciona una opción del menú.")
    elif opcion != "3":
        print("Opción no válida")
```

3.4 Resumen: ¿qué bucle elegir?

- Elige `for` para recorrer los elementos de una secuencia o repetir una acción una cantidad conocida de veces con `range()`.
- Elige `while` para repetir mientras se cumpla una condición, cuando todavía no sabes cuántas vueltas serán necesarias.

En cualquier bucle, comprueba qué hace que avance y qué condición hace que termine. Esto evita repetir trabajo innecesariamente o crear un bucle que no acaba.

Actividades 3

Resuelve los siguientes problemas diseñando el orden de las instrucciones, las condiciones y la repetición necesarias. No uses clases ni excepciones. Cuando un enunciado indique que los datos introducidos son válidos, no hace falta programar su validación.

Actividad 1: control de acceso

Un sistema permite hasta tres intentos para introducir un código. El código correcto es `"4826"`.

Pide el código y comunica si el acceso está autorizado. Si no es correcto, permite intentarlo de nuevo y avisa de cuántos intentos quedan. Después del tercer error, muestra que el acceso se ha bloqueado. Cuando el código sea correcto, no vuelvas a pedirlo.

Prueba el caso en que se acierta en el primer intento, en el último intento y en ninguno. Resuelve el ejercicio usando un bucle y condicionales.

Actividad 2: comprobar si un número es primo

Pide un número entero mayor que `1` y determina si es primo. Un número primo solo se puede dividir exactamente entre `1` y entre sí mismo.

Prueba, si algún número entre `2` y el número anterior al introducido lo divide exactamente. Si encuentras un divisor, el número no es primo, si terminas las comprobaciones sin encontrar ninguno, sí lo es. Muestra el resultado. Comprueba el programa con un número primo, uno compuesto y el `2`, que es el menor número primo.

Actividad 3: máquina expendedora

Una máquina vende un producto que cuesta `170` céntimos. Solicita el valor de cada moneda en céntimos, de una en una; acepta `10`, `20`, `50` y `100`. La persona puede introducir monedas hasta completar el precio.

Si se introduce una moneda no aceptada, informa del problema y vuelve a solicitar una moneda sin aumentar el importe pagado. Cuando se haya alcanzado o superado el precio, indica que entrega el producto y cuánto cambio debe devolver. El programa debe terminar en cuanto la compra se pueda completar.

No hace falta validar que la entrada se pueda convertir a número. Al final, muestra el cambio tanto en céntimos como en euros.

Actividad 4: control de aforo de una sala

Una sala admite como máximo `8` personas. El programa debe llevar la cuenta de cuántas hay dentro. En cada turno, pide una acción: `entra`, `sale` o `fin`.

- Si la acción es `entra` y aún hay plazas, aumenta el aforo ocupado en uno.
- Si la sala está llena, no admitas a nadie más e informa de ello.
- Si la acción es `sale` y hay personas dentro, reduce el aforo ocupado en uno.
- Si la sala está vacía, no permitas que salga una persona y avisa de que no hay nadie dentro.
- Si se escribe `fin`, termina el control e informa de cuántas personas quedan en la sala.

Después de cada acción distinta de `fin`, muestra el aforo actual. Supón que las acciones introducidas son válidas. Prueba entradas y salidas en los límites, como intentar entrar con la sala llena y salir cuando está vacía.

Actividad 5: simulación de operaciones bancarias

Simula el uso de una cuenta bancaria mediante un menú que se repite hasta que se elija `salir`. La cuenta empieza activa, tiene un titular y un saldo inicial aleatorio entre `0 €` y `1.000 €`. Puedes usar el módulo

`random` para generar el saldo. En cada vuelta, pide una operación: `ingresar`, `retirar`, `consultar`, `cerrar cuenta`, `abrir cuenta` o `salir`.

Aplica estas reglas:

- **Ingresar**: solicita una cantidad positiva y súmala al saldo. Solo se puede operar con una cuenta activa.
- **Retirar**: solicita una cantidad positiva. Solo permite la retirada si la cuenta está activa y hay saldo suficiente.
- **Consultar**: muestra el titular, el saldo y si la cuenta está activa o cerrada.
- **Cerrar cuenta**: solo se puede cerrar si está activa y el saldo es `0`.
- **Abrir cuenta**: solo se puede abrir si está cerrada. Al abrirla, el saldo empieza en `0 €`.
- **Salir**: termina el menú y muestra el estado final de la cuenta.

Si una operación no se puede realizar, explica el motivo y vuelve a mostrar el menú. Normaliza el texto de la operación para aceptar mayúsculas y espacios sobrantes. Supón que las cantidades introducidas son números positivos; no hace falta gestionar errores de conversión. Prueba una secuencia que incluya un ingreso, una retirada superior al saldo disponible, una consulta, un intento de cierre con saldo, una retirada que deje el saldo en cero, el cierre, una nueva apertura y la salida.

Actividades extra

Actividad 6: promedio de notas por alumno

Dispones de una lista bidimensional con el nombre de cada alumno y sus tres notas. Cada fila contiene primero el nombre y, a continuación, las notas; por ejemplo:

```
alumnos = [
    ["Ana", 8, 7.5, 9],
    ["Luis", 6, 8, 7],
    ["Marta", 9.5, 10, 8.5],
]
```

Recorre la lista y calcula el promedio de las tres notas de cada alumno. Muestra el nombre y el promedio de cada uno en una tabla clara, con encabezados para las columnas. Presenta los promedios con dos decimales.

Ejemplo de datos y salida:

```
Datos:
Ana: 8, 7.5, 9
Luis: 6, 8, 7
Marta: 9.5, 10, 8.5

Alumno  | Promedio
--------|---------
Ana     | 8.17
Luis    | 7.00
Marta   | 9.33
```

Actividad 7: contar las letras de una palabra

Pide al usuario una palabra y cuenta cuántas veces aparece cada letra. El conteo no debe distinguir entre mayúsculas y minúsculas: por ejemplo, en `"Casa"` la `a` aparece dos veces. Usa un diccionario para guardar cada letra y su cantidad.

Muestra los resultados en una tabla con una columna para la letra y otra para su frecuencia. Cada letra debe aparecer una sola vez en la tabla.

Ejemplo de ejecución:

```
Dígame una palabra: Casa

Letra | Frecuencia
------+-----------
c     | 1
a     | 2
s     | 1
```

Actividad 8: sucesión de valores enteros

Pide un valor inicial, un valor final mayor que el inicial y cuántos valores intermedios se desean. Genera y muestra la lista de valores enteros que empieza en el valor inicial y termina en el final, repartiendo los valores de forma equidistante. Elige valores que permitan obtener saltos enteros. No cuentes los extremos como valores intermedios. Por ejemplo, con los valores `10`, `25` y `2`, la lista debe ser `[10, 15, 20, 25]`.

Ejemplo de ejecución:

```
Dígame el valor inicial: 10
Dígame el valor final: 25
Dígame el número de valores intermedios: 2
La lista correspondiente es [10, 15, 20, 25]
```

Actividad 9: detectar números pares

Pregunta cuántos números se van a introducir y pide esa cantidad de números. Al terminar, indica si se ha escrito al menos un número par. Prueba tanto una serie que incluya un número par como otra en la que todos sean impares.

Ejemplo de ejecución:

```
Dígame cuántos números va a escribir: 5
Dígame el número 1: 17
Dígame el número 2: 5
Dígame el número 3: 7
Dígame el número 4: 22
```

```
Dígame el número 5: 19
Ha escrito algún número par
```

Actividad 10: crear una lista de letras

Pide una palabra o frase y crea una lista con sus caracteres, manteniendo el orden original e incluyendo los espacios. Muestra la lista resultante.

Ejemplo de ejecución:

```
Dígame algo: Una prueba
La lista creada es ['U', 'n', 'a', ' ', 'p', 'r', 'u', 'e', 'b', 'a']
```

Actividad 11: sustituir las vocales de una frase

Pide una frase y una vocal. Crea una nueva frase sustituyendo todas las vocales de la frase original por la vocal indicada. Conserva los espacios y las consonantes, y muestra el resultado. Por ejemplo, con la frase `"tengo una hormiguita"` y la vocal `a`, el resultado empieza por `"tanga ana..."`.

Ejemplo de ejecución:

```
Dígame algo: tengo una hormiguita en la panchita
Dígame una vocal: a
La frase es ahora: tanga ana harmagaata an la panchata
```

Actividad 12: calcular la letra del DNI

Pide el número de un DNI sin letra y calcula la letra correspondiente. Para obtenerla, divide el número entre `23` y usa el resto como índice en esta cadena: `TRWAGMYFPDXBNJZSQVHLCKE`. Muestra el número seguido de su letra.

Ejemplo de ejecución:

```
Dígame su DNI (sin letra): 31415926
Su DNI (con letra) es: 31415926L
```

Actividad 13: convertir un número a números romanos

Pide un número entero positivo inferior a `5000` y muestra su representación en números romanos. Usa los valores `I=1`, `V=5`, `X=10`, `L=50`, `C=100`, `D=500` y `M=1000`. Para simplificar, se permite la notación romana antigua: pueden aparecer hasta cuatro símbolos iguales seguidos. Por ejemplo, `3799` se escribe `MMMDCCLXXXXVIIII`.

Ejemplo de ejecución:

```
Dígame un número: 3799
En números romanos, 3799 se escribe MMMDCCLXXXXVIIII
```

Actividad 14: contar años bisiestos entre dos años

Pide dos años y recorre todos los años del intervalo, incluyendo ambos extremos. Cuenta cuántos son bisiestos y suma el número de días de cada año del intervalo. Un año es bisiesto si es divisible por `400`, o si es divisible por `4` pero no por `100`. Muestra la cantidad de años bisiestos y el total de días. Si el primer año es posterior al segundo, intercambia los valores para recorrer el intervalo en orden.

Ejemplo de ejecución:

```
Dígame un año: 1999
Dígame otro año: 2001
Entre 1999 y 2001 (ambos incluidos) hubo 1 año bisiesto
y en total 1096 días.
```

Actividad 15: jugar a adivinar una clave

Crea una versión sencilla de Master Mind. Pide una longitud entre `2` y `9` y genera una clave secreta de esa longitud formada por cifras distintas. A continuación, pide intentos hasta que se adivine la clave. Después de cada intento, indica cuántas cifras coinciden con la clave tanto en valor como en posición. Cuando se acierten todas, muestra un mensaje de felicitación y termina el juego. Puedes utilizar el módulo `random` para generar la clave.

Ejemplo de ejecución (la clave generada en esta partida es `2431`):

```
Dígame la longitud de la cadena: 4
Intente adivinar la cadena: 1234
Con 1234 ha adivinado 1 valores. Intente adivinar la cadena: 2435
Con 2435 ha adivinado 3 valores. Intente adivinar la cadena: 2431
Con 2431 ha adivinado 4 valores. Felicidades
```

---

4. Estructuras de salto

⬆ Volver al índice

En esta sección veremos dos instrucciones que cambian el recorrido de un bucle: `break` y `continue`. Ambas afectan al bucle más cercano en el que aparecen, pero de forma distinta.

4.1 `break`

`break` termina inmediatamente el bucle más cercano. El programa continúa con la primera instrucción que haya después de ese bucle. No termina el programa completo ni, si hay bucles anidados, los bucles exteriores.

```
for numero in range(100):
    if numero == 10:
        break
    print(numero)
```

En este ejemplo se muestran los números del `0` al `9`. Cuando `numero` vale `10`, se ejecuta `break`, por lo que no se muestra ese número y el bucle acaba.

4.2 `continue`

`continue` no termina el bucle: interrumpe solo la iteración actual. Se omiten las instrucciones que quedan en esa vuelta y el bucle pasa a la siguiente iteración.

```
for numero in range(1, 11):
    if numero % 2 == 0:
        continue
    print(numero)  # solo impares
```

El ejemplo muestra los números impares del `1` al `9`: para cada número par, `continue` salta `print(numero)`, pero el bucle sigue con el siguiente número.

Criterio de diseño

Usa estas instrucciones cuando hagan más claro el recorrido del bucle. La condición que provoca el salto debe ser fácil de identificar: `break` sirve para salir del bucle y `continue` para descartar solo la vuelta actual. Si aparecen muchos saltos, puede resultar más difícil entender qué instrucciones se ejecutan y en qué orden.

---

5. Documentación de programas

⬆ Volver al índice

Documentar un programa consiste en dejar por escrito la información que otras personas —incluida quien lo creó meses después— necesitan para utilizarlo, entenderlo y mantenerlo. La documentación complementa al código: explica su propósito y sus contratos, mientras que el código muestra exactamente cómo se ejecuta.

No toda la documentación debe estar en el mismo lugar. El `README.md` orienta sobre el proyecto completo; los docstrings describen módulos, clases y funciones desde el propio código; las anotaciones de tipo muestran qué datos se esperan; y los comentarios aclaran decisiones que no resultan evidentes al leer las instrucciones. Elegir el lugar adecuado evita repetir información y facilita encontrarla.

Una regla útil es documentar lo que una persona necesita saber y no puede deducir fácilmente del código. Hay que explicar el propósito, los pasos de uso, las restricciones, los resultados y los errores relevantes. No conviene describir línea por línea una operación sencilla: si el código cambia, ese comentario puede quedar desactualizado y confundir.

5.1 README

El archivo `README.md` es la puerta de entrada al proyecto. Debe permitir que alguien que no conoce el código sepa qué resuelve y pueda ponerlo en marcha. Según el tamaño del proyecto, puede incluir:

- **Propósito:** qué problema resuelve y para quién.
- **Requisitos:** versión de Python y dependencias necesarias.
- **Instalación:** cómo preparar el entorno y las dependencias.
- **Ejecución:** comando para iniciar el programa.
- **Uso:** ejemplo de entrada y salida o una descripción de las opciones.
- **Pruebas:** cómo ejecutarlas, si el proyecto las incluye.
- **Configuración:** opciones necesarias, sin publicar contraseñas ni claves.
- **Estructura y mantenimiento:** dónde están las partes principales y cómo contribuir, cuando sea pertinente.

Por ejemplo, no basta con indicar «ejecuta `main.py`» si antes hay que crear un entorno virtual o instalar paquetes. Es preferible escribir los pasos en el orden en que se deben realizar y comprobarlos en una instalación limpia.

Ejemplo de estructura habitual de un proyecto:

```
mi_proyecto/
├── README.md
├── requirements.txt
├── .gitignore
├── src/
│   └── mi_app/
│       ├── __init__.py
│       └── main.py
└── tests/
    └── test_main.py
```

5.2 Docstrings de funciones

Un docstring es una cadena de texto situada al principio de un módulo, una clase o una función. Describe su propósito y forma parte de la información que las herramientas de Python pueden mostrar, por ejemplo mediante `help()`. Para una función, conviene explicar los parámetros, el valor que devuelve y las excepciones que puede producir en condiciones relevantes.

```python
def convertir_kilometros_a_millas(kilometros):
    """Convierte una distancia de kilómetros a millas.

    Args:
        kilometros: Distancia no negativa en kilómetros.

    Returns:
        La distancia equivalente en millas.

    Raises:
        ValueError: Si la distancia es negativa.
```

```
    """
    if kilometros < 0:
        raise ValueError("La distancia no puede ser negativa")
    return kilometros * 0.621371
```

El docstring describe el contrato de la función: qué espera, qué garantiza y qué error comunica cuando no se cumple una condición. Debe coincidir con el código; por ejemplo, no se debe prometer aceptar valores negativos si la función los rechaza.

5.3 Docstrings de clases

El docstring de una clase debe describir qué representa y, cuando no sea obvio, cómo se crea o qué invariantes mantiene. Los métodos públicos también pueden documentarse si tienen reglas de uso propias.

```
class Cuenta:
    """Representa una cuenta con saldo y operaciones básicas."""

    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo
```

5.4 Documentar decisiones

Los comentarios son adecuados para explicar el motivo de una decisión, una restricción externa o un comportamiento que no se deduce del código. Si el motivo es relevante para el mantenimiento, debe actualizarse junto con el código al cambiar esa decisión.

```
# Se conserva el orden de llegada porque el informe se revisa cronológicamente.
eventos = list(eventos_recibidos)
```

5.5 Calidad de la documentación

Una documentación útil permite completar una tarea concreta —por ejemplo, instalar el programa o llamar a una función— sin tener que adivinar pasos. Para conseguirlo, debe ser:

- Exacta y actualizada.
- Clara y organizada según quién la consulta: usuario, desarrollador o persona que mantiene el proyecto.
- Concisa cuando el comportamiento es obvio y detallada cuando hay reglas, límites, efectos secundarios o errores importantes.
- Accionable: permite instalar y ejecutar.
- Verificable: los ejemplos funcionan.
- Coherente con los nombres del código.

Antes de publicar un cambio, revisa si también han cambiado las instrucciones de instalación, los ejemplos, los parámetros o los resultados descritos. Eliminar o corregir información obsoleta es tan importante como añadir información nueva.

5.6 Generación automática

Las herramientas de generación reúnen docstrings y páginas escritas en Markdown para producir documentación fácil de consultar. Cada una se adapta a necesidades distintas: `pydoc` permite empezar rápidamente, Sphinx ofrece más opciones para proyectos Python y MkDocs facilita crear un sitio a partir de páginas Markdown.

1. `pydoc`

Es la opción más sencilla y viene incluida con Python. Puede mostrar la documentación de módulos en el navegador o generar un archivo HTML. Para probarlo, desde la terminal ejecuta:

```
python -m pydoc -b
```

Se iniciará un servidor local y se abrirá el navegador con la documentación de los módulos disponibles. Para generar un archivo HTML con la información de un módulo propio, sitúate en la carpeta desde la que se puede importar y ejecuta:

```
python -m pydoc -w mi_programa
```

El comando crea `mi_programa.html` a partir de los docstrings del módulo.

La generación automática no sustituye la escritura cuidadosa: publica lo que se ha expresado en los docstrings y las páginas, pero no comprueba por sí misma que sus explicaciones sigan siendo ciertas. Revisa los ejemplos y actualiza la documentación cuando cambie el programa.

Actividades 5

Actividad 1: organiza tu día libre

Desarrolla un programa que ayude al usuario a organizar un día libre según su presupuesto, el tiempo disponible y sus preferencias.

El programa deberá mostrar las posibles actividades, como ir al cine, salir a comer, visitar un museo, hacer deporte o quedarse en casa. El usuario podrá elegir varias actividades y el programa comprobará si puede realizarlas teniendo en cuenta el dinero y el tiempo disponibles. Para ello el usuario debe indicar de cuánto tiempo dispone y cuál es su presupuesto.

Cada actividad tendrá un precio y una duración. Si el usuario no dispone de suficiente dinero o tiempo, el programa deberá avisarle y ofrecerle la posibilidad de elegir otra actividad.

Al finalizar, se mostrará un resumen del día con las actividades seleccionadas, el dinero gastado, el tiempo utilizado y el presupuesto restante.

El programa deberá cumplir los siguientes requisitos:

- Utilizar condicionales para comprobar si se puede realizar cada actividad.

- Utilizar bucles para permitir que el usuario seleccione varias actividades.
- Crear funciones para mostrar el menú, comprobar el presupuesto, calcular el tiempo disponible y mostrar el resumen final.
- Validar los datos introducidos para evitar errores.
- Documentar las funciones mediante docstrings y añadir comentarios cuando sean necesarios.

El programa deberá ejecutarse por consola, ser fácil de utilizar y estar organizado en funciones independientes.
